

Bitebimuv ve Mutluet için Çok Bulutlu, Denetlenebilir AI-Ajan Fabrikası

Neyi kurmaya çalışıyoruz ve neden bu kadar zor

İstediğin şey, “tek seferlik bir uygulama” değil; **sürekli çalışan, ihtiyaçları veriyle keşfeden, doğru ürünleri seçen ve seçtiği ürünleri uçtan uca üretebilen** bir “ürün fabrikası”. Bu fabrikanın kritik farkı: “ajanlar yazsın” yaklaşımının tipik zayıflıklarını (hedef sapması, eksik test, güvenlik açıkları, rastgele mimari, kopya-yapıştır entegrasyon, gizli anahtar sızıntısı vb.) **politikalarla, doğrulamalarla ve denetlenebilir iş akışlarıyla** sistematik olarak engellemek.

Bunu “devasa ve güvenilir” yapan şey; aynı anda Kubernetes, MongoDB Atlas, Azure, Google Cloud gibi servisleri kullanmaktan çok, **tüm üretim hattını aynı standarda bağlayan kontrol katmanını (control plane)** doğru kurmak. Bu katman; “hangi ajan neyi, hangi yetkiyle, hangi kanıtla yaptı?” sorusuna her an yanıt veren bir kayıt/audit omurgası üretir.

Bu rapor; WordPress tabanlı bitebimuv.org ve mutluet.org yayınlarının yanında, kullanıcıların mobil/ masaüstü uygulama olarak kullanacağı projeleri “ihtiyaç analizi → ürün seçimi → üretim → dağıtım → izleme” döngüsünde üreten, **projeden sapmayı otomatik yakalayan** bir sistemi; **yüksek güvenlik ve profesyonel SDLC** şartlarıyla tarif eder.

Ürün stratejisi: İhtiyaçları seçen mekanizma ve “en çok ihtiyaç duyulan” çıktılar

“İnsanların en çok ihtiyaç duyduğu uygulamalar” konusu, tek bir doğru liste değil; **hedef kitlenin, coğrafyanın ve eldeki kanıtların** (site analitiği, destek talepleri, gönüllü/bağış davranışı, kurum talepleri, sosyal veri) birleşiminden çıkan bir sıralamadır. Bu yüzden sistemin omurgası şu olmalı: **kanıt üretmeden karar vermeyen** bir ihtiyaç motoru.

Bu ihtiyaç motorunun pratikte üreteceği “yüksek olasılıklı” ilk ürün dalgaları (Mutluet’in mevcut yönüyle de uyumlu) genellikle şu kümelerde toplanır:

Birincisi, “yardım ve kaynak bulunabilirliği”. Kurum rehberi, doğrulanmış hizmet noktaları, kriz anı yönlendirme, yakın çevrede güvenilir destek arama; **tekil içerik değil, yaşayan veri** gerektirir. İkincisi, “gönüllülük ve bağışın operasyonu”. Etkinlik keşfi, gönüllü saatleri, bağış akışları, şeffaf raporlama gibi fonksiyonlar, STK’ların pratik ihtiyacıdır. Üçüncüsü, “güvenli iletişim/bağlantı”. Gerçek zamanlı mesajlaşma, gerekirse görüntülü görüşme gibi fonksiyonlar; topluluk bağını ve vaka yönetimini güçlendirir. Dördüncüsü, “içerik ve farkındalık yayıncılığı”. WordPress burada güçlüdür ama WordPress’in içinden “platform işlevleri”ni yönetmek için entegrasyon gerekir; WordPress REST API bunu standardize eder ve **Application Passwords** ile revokable (geri alınabilir) entegrasyon kimliği sağlar. ¹

Bu raporun geri kalanı, “hangi uygulama lazım?” sorusunu **otomatik ve denetlenebilir biçimde** cevaplayan sistemi kurmaya odaklanır: ihtiyaç motoru her döngüde bir “ürün backlog’u” üretir; üretim hattı da backlog’u uygulamaya dönüştürür.

Referans mimari: Çok bulut, Kubernetes, WordPress ve mobil/ masaüstü panel

Kontrol katmanı ve iş yükü katmanı ayrımı

Bu ölçekte güvenilirlik için mimariyi ikiye ayır:

Kontrol katmanı; ajan orkestrasyonu, politika motorları, CI/CD, GitOps, gizli anahtar yönetimi, audit log, gözlemlleme. İş yükü katmanı; Mutluet/Bitebimuv uygulamaları, API'ler, WordPress siteleri, arka plan işler, arama/vektör, mesajlaşma.

Bu ayrım, “her şeyi her bulutta çalıştırma” isteğini yönetilebilir hale getirir: uygulamalar taşınabilir olur; kontrol katmanı standartları zorlar.

GitOps ile dağıtım: Argo CD veya Flux

Kubernetes tarafında “güvenilir devasa sistem” genellikle **GitOps** ile kurulur: cluster ve uygulama manifestleri Git'te “tek gerçek kaynak” olur, cluster o Git'i deklaratif şekilde uygular. OpenGitOps, GitOps'u CNCF altında vendor-nötr prensiplerle tanımlamak için çalışır; bu, araç seçsen bile metodolojiyi sabit tutmanı sağlar. ²

Uygulama dağıtımı için Argo CD yaklaşımı, özellikle çoklu cluster yönetiminde güçlüdür: CLI ile cluster ekleme ve Argo CD'nin gerekli kaynakları kurması, operatör işini hızlandırır. ³

Argo CD ayrıca cluster erişim bilgilerini “secret” olarak yönetir ve deklaratif kurulumda bu secret'ların etiketlenmesini bekler; bu, “hangi cluster'a ne yetki verildi?” sorusunu Git geçmişiyle denetlenebilir kılar. ⁴

Flux ise GitOps Toolkit bileşenleri (source-controller, kustomize-controller, helm-controller vb.) ile daha modüler bir yaklaşım sunar; örneğin source-controller Git/OCI/Helm kaynaklarını artefaktlaştırıp cluster içi tüketilebilir hale getirir. ⁵

Tercih önerisi: “tek panelden devasa görünürlük” istiyorsan Argo CD ile başla; “her cluster kendi kendini yönetsin” istiyorsan Flux düşün.

Kubernetes güvenlik standartları: Pod Security + RBAC + NetworkPolicy

Bu çapta otomasyon, güvenlik standartlarını “varsayılan” yapmazsan dağılır.

Pod düzeyi güvenlikte Kubernetes **Pod Security Standards** üç profil tanımlar: Privileged, Baseline, Restricted. Baseline bilinen privilege escalation yollarını engellemeyi hedefler; Restricted ise pod hardening best-practice'lerine yaklaşır. ⁶

Bunları uygulamak için Kubernetes'te **Pod Security Admission** yerleşik admission controller olarak kullanılabilir ve namespace seviyesinde enforce/audit/warn modlarıyla kademeli devreye alınır. ⁷

Yetkilendirme tarafında Kubernetes RBAC; Role/ClusterRole ve RoleBinding/ClusterRoleBinding ayrımıyla “namespace içi” ve “cluster geneli” yetkileri ayırır. Bu ayrım, ajanların ve CI sistemlerinin sadece gereken kapsama erişmesini sağlar. ⁸

Ağ izolasyonunda Kubernetes **NetworkPolicy**, pod'lar arası ingress/egress trafiğini etiket tabanlı kurallarla kısıtlamayı standardize eder; “policy yoksa her şey açık” olmaması için default-deny desenleriyle ilerlenir. ⁹

Politika motoru: Kyverno veya Gatekeeper

Kubernetes'in admission aşamasında "bu manifest kabul edilir mi?" sorusunu otomatik cevaplamak için bir politika motoru şart.

Kyverno; Kubernetes-native policy engine olarak kendini konumlar, YAML ve CEL ile politika yazmayı teşvik eder, admission controller olarak validate/mutate/generate/cleanup yetenekleri sunar ve ayrıca container image doğrulama gibi supply-chain odaklı kontrolleri hedefler. ¹⁰ Gatekeeper ise OPA ekosisteminde "Rego policy controller" olarak Kubernetes admission politikalarını custom resource'larla yönetmeye odaklanır. ¹¹

Pratik öneri: ekiplerin Kubernetes YAML'ine aşinalığı yüksekse Kyverno ile onboarding daha hızlı olur; Rego standardın varsa Gatekeeper tercih edilebilir.

Gizli anahtar omurgası: External Secrets Operator + bulut secret manager

Senin senaryonda "kullanıcı sadece `.env` doldursun" hedefi aslında şu anlama gelmeli: **kullanıcı bir kere bootstrap `.env` verir, sistem tüm secret'ları doğru yere taşır ve bundan sonra runtime secret'lar Kubernetes'e otomatik akar.**

External Secrets Operator (ESO), AWS Secrets Manager / HashiCorp Vault / Google Secret Manager / Azure Key Vault gibi dış secret kaynaklarını okuyup Kubernetes Secret içine senkronize eden bir operatör yaklaşımıdır. ¹²

Azure Key Vault provider dokümantasyonu, AAD Pod Identity'nin deprecated olduğunu ve Workload Identity yaklaşımının önerildiğini vurgular; bu, "uzun ömürlü secret yerine identity ile yetkilendirme" prensibini güçlendirir. ¹³

Veri katmanı: PostgreSQL + MongoDB Atlas (vektör dahil) hibriti

Mutluet tarzı işlerde tipik desen: transactional veriler (kullanıcı, bağlı, etkinlik, mesaj meta) için Postgres; esnek doküman/veri keşfi ve AI RAG için doküman store + vektör arama.

MongoDB Atlas, **Vector Search** ile vektör verisini "operasyonel veriyle aynı yerde" tutup semantik arama ve hibrit arama yapmayı hedefler; RAG/agentik sistemleri desteklemek için konumlanır. ¹⁴

MongoDB'nin Atlas Kubernetes Operator yaklaşımı, Atlas kaynaklarını Kubernetes custom resource'ları üzerinden yönetmeye ve CI/CD ile Atlas'ı aynı kontrol düzlemine bağlamaya odaklanır. ¹⁵

Alternatif denetim: Eğer Postgres'te pgvector kullanıyorsan, vektör ihtiyacını Postgres'te de çözebilirsin; ama MongoDB Atlas Vector Search "hibrit arama + operational data birlikte" vaadini daha doğrudan sunar. Kararı "veri şekli + sorgu tipi + takım yetkinliği + maliyet" belirlemeli.

WordPress: içerik motoru, ama platformun parçası

WordPress'i sadece "site" değil, fabrikanın bir UI yüzü gibi kurgulamak için iki kritik nokta var:

WordPress REST API, uç noktaları standardize eder; içerik (posts/pages/media) ve yönetim aksiyonları API ile yapılabilir. ¹⁶

WordPress **Application Passwords**, üçüncü taraf entegrasyonlara revokable kimlik verir; ana şifreyi paylaşmadan, uygulama bazlı credential üretir ve HTTPS üzerinden Basic Auth ile REST API çağrılarını destekler. ¹⁷

Bu sayede “AI üretim fabrikası”, WordPress’e otomatik içerik basabilir, medya yükleyebilir, sayfaları güncelleyebilir; ayrıca WordPress admin paneli içine bir “Bitebimuv Factory” eklentisi koyarak üretim işlerini tetikleyebilir.

macOS uygulaması ve paketleme: Tauri ile paneli uygulamaya çevirme

Paneli “mobilde de çalışacak” yapmak için temel çözüm: responsive web app (PWA) + opsiyonel native sarmalayıcı.

Tauri; web UI’yi yerel uygulama kabuğuna sarıp `.app/.dmg` üreterek macOS dağıtımı yapabilir; macOS’ta codesign/notarization süreçlerini açıkça tarif eder ve ortam değişkenleriyle (APPLE_SIGNING_IDENTITY vb.) CI/CD’de imzalama/notarization akışına izin verir. ¹⁸

Önemli pratik detay: Notarization için Apple tarafında kimlik bilgileri gerekir; Tauri dokümantasyonu App Store Connect API veya Apple ID ile doğrulama seçeneklerini listeler. ¹⁹

Ajan fabrikası: Uçtan uca denetlenen iş akışları ve “projeden sapma” kilidi

Ajan rolleri: tek süper-ajan yerine, görev hattı

Güvenilirlik için “her işi yapan tek ajan” yerine, her adımı farklı ajan(lar)a böl:

Ürün keşif ajanı; veri toplar, kanıt haritası çıkarır, ihtiyacı tanımlar. Ürün yönetimi ajanı; ihtiyaçları skorlama (etki, aciliyet, maliyet, risk), “bu sprintte bu ürün” karar taslağı üretir. Mimari ajan; seçilen ürün için hedef mimari, veri modeli, entegrasyon matrisi çıkarır. Kod üretim ajanı; monorepo şablonlarından kod üretir. Test ajanı; unit/integration/e2e test kapsamını zorlar. Güvenlik ajanı; secret sızıntısı, bağımlılık güvenliği, container güvenliği, policy kontrolü çalıştırır. Dağıtım ajanı; GitOps PR’larını açar, Argo/Flux senkronunu izler. Gözlemlleme ajanı; SLO/SLI, log/trace/metric alarmlarını kurar ve geri besleme üretir.

“Projeden sapma”yı engellemenin pratik yolu: her aşamada bir **spesifikasyon sözleşmesi** ve bunun otomatik doğrulaması.

Spesifikasyon sözleşmesi: değişmeyen hedef, değişen uygulama

Her ürün için tek bir “kaynak-of-truth spec” dosyası üret:

`/specs/<product>/spec.yaml` gibi.

Bu spec, şu bölümleri içerir: amaç, kullanıcı tipleri, kapsam dışı maddeler, işlevler (acceptance criteria), veri sınıflandırması (KVKK/PII), entegrasyonlar, SLO hedefleri, maliyet sınırı, release kriterleri.

Sonra şu kuralı koy: “Ajanlar kodu değiştirebilir ama spec’e aykırı davranamaz.” Bu kuralı Kyverno/Gatekeeper tarafında değil, **CI içinde** enforce edersin: PR’da yapılan değişiklikler, spec maddeleriyle otomatik karşılaştırılır; testler + policy check + “spec compliance check” geçmeden merge yok.

Denetim kanıtları: SLSA provenance + imzalı artefact

“Her aşama denetlensin” ifadesini teknik olarak güçlü yapan şey: **kanıt üretmek**.

SLSA, provenance'i; bir artefact'ın nerede/nasıl/ne girdilerle üretildiğini doğrulanabilir bir attestation olarak tarifler ve "builder.id, buildDefinition, runDetails" gibi alanlarla build platformuna güven kökünü bağlar. ²⁰

Sigstore/cosign, in-toto attestations doğrulama tarafında `cosign verify-attestation` gibi komutlarla ve predicate doğrulamada politika dilleri (CUE/Rego) ile "fail closed" tasarımını vurgular. ²¹

Bu kombinasyonla yaklaşım şu olur: her container image için SBOM + provenance üret, cosign ile imzala; cluster admission katmanında (Kyverno/Gatekeeper) "imzasız image çalışmaz" kuralı koy.

MCP ve ajan araç erişimi: bağlayıcı standardizasyon

Birden çok sistemle entegre ajanların sürdürülebilir olması için "bağlantı standardı" gerekir. Anthropic'in **Model Context Protocol (MCP)** yaklaşımı, uygulamaların LLM'lere context ve tool sağlamasını standardize eden açık protokol olarak konumlanır. ²²

MCP connector dokümanı, Messages API üzerinden uzak MCP server'lara bağlanmayı, OAuth bearer token desteğini ve sınırlamaları (ör. yalnız tool calls desteklenmesi, server'ın public HTTP olması) açıkça belirtir. ²³

Buradaki stratejik sonuç: "Slack/GitHub/Google Drive/Calendar gibi" kaynaklara ajan erişimini, tek tek özel entegrasyon yazmak yerine MCP server'lar ile standardize edebilir; fakat bunu üretimde kullanırken **yetki sınırları, audit ve token yönetimi** mutlaka control plane tarafından yönetilmelidir.

Uygulama planı: Sıfırdan üretime, `.env` ile bootstrap edilen profesyonel hat

Aşağıdaki plan, "kullanıcı en sonda sadece `.env` hazırlasın" hedefine en yakın gerçekçi düzeni verir: ilk kurulumda bir bootstrap `.env` ile kimlikler tanımlanır; sistem secret manager + Kubernetes'e taşır; sonra ilerleyen her şey otomatik akar.

Başlangıç kurulumu: tek komutla yerel ve cloud bootstrap

Yerelde MacBook üzerinde:

Docker + Kubernetes yerel cluster (ör. kind) + CI simülasyonu. Bu sayede ajanların "üretim hattını" canlıya çıkmadan test edersin.

Cloud'da:

Bir "platform cluster" seç; Azure kredin olduğu için AKS güçlü aday; GCP tarafı ikinci cluster olarak sonra eklenir. Uygulama cluster'ları ilk etapta tek region + tek bulut, daha sonra multi-region/multi-cluster genişler.

Argo CD/Flux kurulumu: GitOps repo yapısı oluştur, cluster bootstrap manifestlerini oraya koy.

Secret yönetimi: `.env` → secret manager → ESO → Kubernetes

Bootstrap `.env` yalnızca şu sınıfları içermeli: - Bulut hesabı kimliği (service principal / workload identity tanımları) - Git provider token'ları - AI provider API key'leri (OpenAI/Anthropic/Gemini vb.) - Stripe, SendGrid gibi dış servis anahtarları - WordPress application password veya OAuth client secret'ları

Sonra: - Secret'ları **Azure Key Vault / GCP Secret Manager** gibi sistemlere yaz - ESO ile Kubernetes'e senkronla ¹² - Uygulama pod'ları yalnız K8s Secret görsün; `.env` üretim ortamına kopyalanmasın

Bu yaklaşım, "ajanlar repo içine secret yazdı" kazasını ciddi ölçüde azaltır.

WordPress entegrasyonu: içerik ve admin tetikleme

WordPress tarafında iki parça:

"Headless/content" erişimi: REST API ile post/page/media üretimi. WordPress REST API referansı, endpoint'leri ve kapsamı tanımlar. ¹⁶

Kimlik doğrulama: Application Passwords ile revokable credential üretip API çağrılarını HTTPS üzerinden Basic Auth ile yaparsın; WordPress bu yöntemi ana şifreyi paylaşmaya alternatif olarak konumlar. ¹⁷

Admin panel eklentisi: WP admin menüsüne "Factory" sayfası eklenir. Buradan: - ihtiyaç analizi çalıştır - "yeni ürün" başlat - CI/GitOps durumlarını izle - yayın/dağıtım raporu üret

Panelin mobil ve macOS uygulaması: tek kod tabanı, çok çıktı

Paneli bir web uygulaması olarak tasarlayıp: - mobilde responsive çalıştır (gerekirse PWA) - macOS'ta Tauri ile paketle

Tauri'nin macOS codesign/notarization akışı; sertifika kimliği, environment variable'larla signing identity ve App Store Connect/Apple ID ile notarization adımlarını tarif eder. ¹⁸

Ajan zamanlaması: "hangi saatlerde ne çalışmalı" önerisi

Bu kısım, maliyet ve operasyon ritmi için uygulanabilir bir çerçeve. Saatler Europe/Istanbul (senin zaman dilimin) varsayımıyla:

Gece döngüsü; veri toplama ve batch analiz. 02:00–05:00 arası araştırma ajanı; site analitiği, formlar, ticket/backlog, sosyal sinyaller gibi kanallardan özet çıkarır. 05:00–06:00 arası ürün yönetimi ajanı; skorlar, "ilk 3 ihtiyaç" shortlist üretir ve karar kaydı taslağı yazar.

Gündüz döngüsü; üretim ve doğrulama. 10:00–12:00 arası mimari ajan + tasarım ajanı; spec günceller, mimari diyagram + veri modeli üretir. 13:00–17:00 arası kod üretim ajanı; şablonlardan modül üretir. 17:00–19:00 arası test + güvenlik ajanı; policy check, RBAC/netpol uyumu, container hardening, supply chain attestation süreçlerini çalıştırır (Pod Security Standards ve admission yaklaşımına uyumluluk kontrolü dahil). ²⁴

20:00 sonrası; GitOps PR'ları açılır, Argo CD sync izlenir, canary/rollback stratejileri devreye girer.

Kritik üretim kuralı: "Ajanlar 7/24 yazabilir ama merge/deploy 'gates' saatlerini sabitle." Böylece operasyonel sürpriz azalır.

`.env` şablonu ve platform linkleri: kullanıcıya bırakılacak tek şey

Aşağıdaki şablon; "bootstrap `.env`" mantığıyla, **gerçek secret değerlerini değil placeholder'ları** içerir. Bu dosya yalnızca ilk kurulumda kullanılır; daha sonra secret manager'lara taşınır.

```

# =====
# CORE
# =====
ENV=prod
PROJECT=bitebimuv-factory
REGION_PRIMARY=eu-west-1

# =====
# GIT / CI
# =====
GITHUB_PAT=__FILL__
GITHUB_OAUTH_CLIENT_ID=__FILL__
GITHUB_OAUTH_CLIENT_SECRET=__FILL__

# =====
# AI PROVIDERS
# =====
OPENAI_API_KEY=__FILL__
ANTHROPIC_API_KEY=__FILL__
GEMINI_API_KEY=__FILL__
PERPLEXITY_API_KEY=__FILL__

# =====
# AZURE
# =====
AZURE_TENANT_ID=__FILL__
AZURE_SUBSCRIPTION_ID=__FILL__
AZURE_CLIENT_ID=__FILL__
AZURE_CLIENT_SECRET=__FILL__
AZURE_KEY_VAULT_URI=__FILL__
AZURE_COMMUNICATION_CONNECTION_STRING=__FILL__

# =====
# GOOGLE CLOUD
# =====
GCP_PROJECT_ID=__FILL__
GCP_WORKLOAD_IDENTITY_PROVIDER=__FILL__
GCP_SERVICE_ACCOUNT_EMAIL=__FILL__

# =====
# MONGODB ATLAS
# =====
MONGODB_ATLAS_PUBLIC_KEY=__FILL__
MONGODB_ATLAS_PRIVATE_KEY=__FILL__
MONGODB_ATLAS_ORG_ID=__FILL__
MONGODB_ATLAS_PROJECT_ID=__FILL__

# =====
# WORDPRESS (bitebimuv.org / mutluet.org)
# =====

```

```
WP_MUTLUET_URL=https://mutluet.org
WP_BITEBIMUV_URL=https://bitebimuv.org
WP_API_USERNAME=__FILL__
WP_APP_PASSWORD=__FILL__ # WordPress Application Password (revokable) 25

# =====
# PAYMENTS / EMAIL / MEDIA / OBSERVABILITY
# =====
STRIPE_SECRET_KEY=__FILL__
STRIPE_WEBHOOK_SECRET=__FILL__
SENDGRID_API_KEY=__FILL__
CLOUDINARY_CLOUD_NAME=__FILL__
CLOUDINARY_API_KEY=__FILL__
CLOUDINARY_API_SECRET=__FILL__
SENTRY_DSN=__FILL__
```

Platform linkleri (hesap/anahtar ekranlarına hızlı erişim için) — URL'leri burada “kod bloğu” içinde veriyorum:

```
Azure Portal: https://portal.azure.com
Azure Key Vault: https://learn.microsoft.com/azure/key-vault/ (Key Vault kavramı için)
Google Cloud Console: https://console.cloud.google.com
MongoDB Atlas: https://cloud.mongodb.com
MongoDB Atlas Kubernetes Operator: https://www.mongodb.com/docs/atlas/operator/stable/ 26
External Secrets Operator: https://external-secrets.io 27
Argo CD docs: https://argo-cd.readthedocs.io 3
OpenGitOps: https://opengitops.dev/about 2
WordPress Application Passwords: https://developer.wordpress.org/advanced-administration/security/application-passwords/ 25
Tauri macOS signing: https://tauri.app/distribute/sign/macos/ 28
Anthropic MCP: https://docs.anthropic.com/en/docs/mcp 22
```

Kritik riskler ve profesyonel “tamlik” kontrol listesi

Bu ölçekte en büyük risk, “çok servis kullanmak” değil; **yetki genişlemesi ve kontrolsüz otomasyon**. Kubernetes RBAC'in namespace/cluster ayrımı, yanlış yapılandırmada ya çok kısıtlı kalır ya da tehlikeli şekilde genişler; Kubernetes dokümanı özellikle güç bağıışı (ör. tüm service account'lara cluster-admin) gibi durumlara karşı uyarır. 8

Pod güvenliği tarafında, Pod Security Standards + admission controller ile kademeli enforce yapılmazsa, ajanların ürettiği manifestler farklı güvenlik seviyelerinde dağılır; Kubernetes'in built-in enforcement yaklaşımı bu standardizasyonu hedefler. 29

Ağ izolasyonu yoksa, servisler “default açık” kalır; NetworkPolicy bu yüzden temel güvenlik çizgisidir. 9

Supply-chain tarafında “ajan kod yazdı ve deploy etti” modeli; provenance ve attestation olmadan denetlenebilir değildir. SLSA provenance modeli ve Sigstore attestation doğrulama prensipleri, bu boşluğu kapatmak için tasarlanmıştır. ³⁰

Son olarak macOS paketleme hedefinde, codesign/notarization adımları Apple hesap/sertifika süreçlerine bağlıdır; Tauri dokümantasyonu bu bağımlılığı açıkça belirtir. ¹⁹

Bu yüzden “profesyonel ve eksiksiz” tanımını şu şekilde operationalize et:
Spec sözleşmesi var mı? Test kapısı var mı? Policy-as-code var mı? Secret sızıntısı denetimi var mı?
Provenance + imzalı artefact var mı? GitOps üzerinden geri alınabilir dağıtım var mı? WordPress entegrasyonu revokable kimlikle mi? ³¹

¹ ¹⁷ ²⁵ ³¹ Application Passwords – Advanced Administration Handbook | Developer.WordPress.org
https://developer.wordpress.org/advanced-administration/security/application-passwords/?utm_source=chatgpt.com

² About | OpenGitOps
https://opengitops.dev/about?utm_source=chatgpt.com

³ Cluster Management - Argo CD - Declarative GitOps CD for Kubernetes
https://argo-cd.readthedocs.io/en/stable/operator-manual/cluster-management/?utm_source=chatgpt.com

⁴ Declarative Setup - Argo CD - Declarative GitOps CD for Kubernetes
https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/?utm_source=chatgpt.com

⁵ Source Controller | Flux
https://fluxcd.io/docs/components/source/?utm_source=chatgpt.com

⁶ ²⁴ Pod Security Standards | Kubernetes
https://v1-34.docs.kubernetes.io/docs/concepts/security/pod-security-standards/?utm_source=chatgpt.com

⁷ ²⁹ Enforce Pod Security Standards by Configuring the Built-in Admission Controller | Kubernetes
https://kubernetes.io/docs/tasks/configure-pod-container/enforce-standards-admission-controller?utm_source=chatgpt.com

⁸ Using RBAC Authorization | Kubernetes
https://kubernetes.io/docs/reference/access-authn-authz/rbac/?utm_source=chatgpt.com

⁹ Network Policies | Kubernetes
https://v1-33.docs.kubernetes.io/docs/concepts/services-networking/network-policies/?utm_source=chatgpt.com

¹⁰ Introduction | Kyverno
https://kyverno.io/docs/introduction/?utm_source=chatgpt.com

¹¹ OPA Gatekeeper | Open Policy Agent
https://www.openpolicyagent.org/ecosystem/entry/gatekeeper?utm_source=chatgpt.com

¹² ²⁷ GitHub - external-secrets/external-secrets: External Secrets Operator reads information from a third-party service like AWS Secrets Manager and automatically injects the values as Kubernetes Secrets.
https://github.com/external-secrets/external-secrets?utm_source=chatgpt.com

¹³ Azure Key Vault - External Secrets Operator
https://external-secrets.io/v0.19.2/provider/azure-key-vault/?utm_source=chatgpt.com

¹⁴ MongoDB Vector Search Overview - Atlas - MongoDB Docs
https://www.mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/?utm_source=chatgpt.com

¹⁵ MongoDB Atlas Operator For Kubernetes | MongoDB
https://www.mongodb.com/en-us/products/integrations/kubernetes/atlas-kubernetes-operator?utm_source=chatgpt.com

16 Reference – REST API Handbook | DeveloperWordPress.org

https://developer.wordpress.org/rest-api/reference/?utm_source=chatgpt.com

18 19 28 macOS Code Signing | Tauri

https://tauri.app/distribute/sign/macos/?utm_source=chatgpt.com

20 30 SLSA • Provenance

https://slsa.dev/spec/v1.0-rc1/provenance?utm_source=chatgpt.com

21 In-Toto Attestations - Sigstore

https://docs.sigstore.dev/cosign/verifying/attestation/?utm_source=chatgpt.com

22 Model Context Protocol (MCP) - Anthropic

https://docs.anthropic.com/en/docs/mcp?utm_source=chatgpt.com

23 MCP connector - Anthropic

https://docs.anthropic.com/en/docs/agents-and-tools/mcp-connector?utm_source=chatgpt.com

26 Atlas Kubernetes Operator - Atlas Kubernetes Operator - MongoDB Docs

https://www.mongodb.com/docs/atlas/operator/stable/?utm_source=chatgpt.com